Universidad de La Habana

Facultad de Matemática y Computación

Ciencia de la Computación

Bases de Datos II — Ingeniería de Software

Curso 2026 - 2027

INFORME

Diseño de la Base de Datos

Gestión para la generación automática de planes de entrenamiento

Nombre del equipo: Base de Batos

Integrantes

No.	Nombre completo	Grupo	Nick de Telegram	Usuario de GitHub
1	Bryan Ernesto Moya Aquino	C-312	@BM0ya	el-M0ya
2	Héctor Duarte Vázquez	C-311	@hduartev	HectorDuarte911
3	Fabio Hernández Piloto	C-311	@FabioPilot	Fabio-Pilot
4	Gabriel Pérez Suárez	C-312	@GabrielPS1016	GaboPS1016
5	Alex Leonardo Cuervo Grillo	C-311	@cuervogrillo	AlexCuervo
6	Alejandro Lopez Castro	C-312	@AlejandroL04	Kaik0405

Resumen

El presente informe documenta el proceso de análisis, diseño y valoración teórica de la base de datos correspondiente al proyecto *Gestión para la generación automática de planes de entrenamiento*, en el marco de las asignaturas Bases de Datos II e Ingeniería de Software.

El documento comienza con un análisis y reformulación enriquecedora de los requerimientos funcionales e informacionales del sistema, a partir de los cuales se identifica el conjunto de entidades, atributos e interrelaciones que sustentan el dominio *planes de entrenamiento*. Sobre esa base se construye el Modelo Conceptual Relacional-Extendido (MERX), presentado en una única figura que reúne el modelo completo.

A continuación se especifican las restricciones de integridad del modelo, organizadas en cinco familias: integridad de entidad, integridad de dominio, integridad referencial, integridad semántica y de negocio, e integridad temporal. Para cada restricción se indica además el mecanismo concreto de implantación en el sistema gestor de bases de datos.

El paso del MERX al esquema relacional produce 18 relaciones con 28 claves foráneas, unificando la persistencia transaccional y la estructura variable de bloques mediante tipos de datos nativos `JSONB` en PostgreSQL. La sección de valoración de la corrección del diseño comprueba, de forma explícita y algoritmo a algoritmo, que dicho esquema cumple la Tercera Forma Normal, la propiedad de preservación de dependencias funcionales (PPDF) y la propiedad de join sin pérdida de información (PLJ), apoyándose en el Lema de Ullman para las uniones por clave primaria y en el algoritmo de la tabla para un caso desarrollado. Se demuestra además que cada dependencia del cubrimiento minimal es irredundante, y se identifican las relaciones con claves candidatas alternativas. Se analizan asimismo las anomalías de inserción, eliminación y actualización que el diseño evita, así como las tres redundancias controladas que quedan protegidas por disparadores, y se establece la correspondencia entre cada requerimiento del sistema y las relaciones del esquema que lo soportan.

Se concluye que el diseño propuesto es correcto, se encuentra normalizado en FNBC y constituye una base sólida para las etapas posteriores de implementación.

Índice

1. Introducción **4**

2. Análisis y reformulación enriquecedora de los requerimientos funcionales e informacionales del sistema **4**

2.1. Requerimientos funcionales 4

2.1.1. Gestión de usuarios, roles y sedes 5

2.1.2. Programas de entrenamiento y grupos musculares 5

2.1.3. Banco de ejercicios y su clasificación 5

2.1.4. Autorización de entrenadores e inscripción de clientes 6

2.1.5. Generación y composición de rutinas 6

2.1.6. Asignación de rutinas y ejecución de sesiones 6

2.1.7. Consultas y reportes 7

2.1.8. Presentación y exportación de la información 7

2.2. Requerimientos no funcionales 7

2.3. Requerimientos informacionales 8

2.3.1. Gestión de usuarios, roles y sedes 8

2.3.2. Programas de entrenamiento y catálogos 8

2.3.3. Banco de ejercicios 9

2.3.4. Autorización e inscripción 9

2.3.5. Generación y composición de rutinas 9

2.3.6. Validación 9

2.3.7. Asignación y ejecución 10

2.3.8. Solicitudes de ajuste 10

2.3.9. Soporte de consultas y reportes 10

3. Modelo Conceptual Relacional-Extendido (MERX) **11**

3.1. Esquema relacional derivado del MERX 11

3.2. Estructura variable de bloques de la rutina 16

4. Especificación de las restricciones de integridad establecidas y/o detectadas **16**

4.1. Integridad de entidad 16

4.2. Integridad de dominio 16

4.3. Integridad referencial 17

4.4. Integridad de unicidad 18

4.5. Nulabilidad 18
4.6. Restricciones de cardinalidad por interrelación 19
4.7. Restricciones semánticas y de negocio 20
4.8. Restricciones temporales 21
4.9. Tratamiento de la estructura variable de bloques 22
4.10. Mecanismos de implantación de las restricciones 22

5. Valoración de la corrección del diseño de la base de datos 23
5.1. Objetivo y metodología 23
5.2. Universo de atributos y dependencias funcionales 23
5.3. Cubrimiento minimal 24
5.4. Claves candidatas y formas normales 26
5.5. Propiedad de join sin pérdida de información 26
5.6. Preservación de las dependencias funcionales 29
5.7. Anomalías evitadas 29
5.8. Alcance de la validación formal 30
5.9. Correspondencia entre requerimientos y relaciones 30
5.10. Criterios de evaluación aplicados 31
5.11. Riesgos y mitigaciones 32
5.12. Conclusión de la valoración 32

6. Conclusiones 32

1. Introducción

Este informe documenta el trabajo de análisis, diseño y valoración de la base de datos del proyecto *Gestión para la generación automática de planes de entrenamiento*, dentro del marco de las asignaturas Bases de Datos II e Ingeniería de Software.

El problema de negocio se sitúa en el ámbito de la gestión de gimnasios. El sistema debe administrar el banco de ejercicios disponibles, los programas de entrenamiento y los grupos musculares y tipos de ejercicio que éstos abarcan, los entrenadores responsables de crear y validar rutinas, y los clientes que ejecutan dichas rutinas. A ello se añade la necesidad de que las rutinas se adapten a criterios preestablecidos —nivel de intensidad, tipo de ejercicio y distribución de los grupos musculares cubiertos a lo largo del ciclo de entrenamiento—, así como la posibilidad de que un jefe de sala revise y apruebe las rutinas generadas, o bien indique la confección de otra bajo sus propios criterios.

El informe se organiza de la siguiente manera. La sección 2 reformula los requerimientos en dos dimensiones complementarias: los requerimientos funcionales, que expresan las operaciones que el sistema debe realizar, y los requerimientos informacionales, que describen la estructura de los datos necesaria para soportarlas. La sección 3 presenta el MERX en una sola figura, que reúne la superentidad de usuarios y sus especializaciones, los catálogos del dominio y el banco de ejercicios, la autorización de entrenadores y la inscripción de clientes, la generación y composición de las rutinas, el ciclo de validación, asignación y ejecución, y el proceso de solicitud de ajuste. Para no saturar el diagrama, los atributos no se representan en la figura: figuran en el esquema relacional de la sección 3.1.

La sección 4 especifica las restricciones de integridad del modelo en cinco familias, indicando para cada una el mecanismo de implantación utilizado. La sección 5 realiza la valoración formal de la corrección del diseño mediante la demostración de la 3FN, la comprobación de la PLJ por el Lema de Ullman y la verificación de la PPDF; incluye además el análisis de anomalías y la correspondencia entre requerimientos y relaciones del esquema. Por último, la sección 6 presenta las conclusiones del trabajo.

Este informe fundamenta una arquitectura de datos coherente y teóricamente sustentada, y sirve de base para las etapas posteriores de construcción del esquema físico, de implementación y de pruebas.

2. Análisis y reformulación enriquecedora de los requerimientos funcionales e informacionales del sistema

El enunciado del problema, si bien delimita con precisión el dominio de interés, enuncia los requerimientos en forma narrativa y no estructurada. La reformulación enriquecedora consiste en descomponer esa narrativa en un conjunto de requerimientos atómicos, verificables y trazables, y en completarlos con las decisiones de modelado que la narrativa deja implícitas pero que el diseño de la base de datos necesita para poder construirse.

2.1. Requerimientos funcionales

Los requerimientos funcionales (RF) describen las operaciones, procesos y reglas de negocio que el sistema debe soportar. Se organizan por subsistemas, siguiendo la estructura modular de la solución.

2.1.1. Gestión de usuarios, roles y sedes

RF-01 El sistema debe permitir la autenticación de los usuarios del sistema mediante credenciales (correo electrónico y contraseña).

RF-02 El sistema debe gestionar los roles de los usuarios y restringir las funcionalidades disponibles según el rol: administrador, entrenador, jefe de sala y cliente.

RF-03 El sistema debe permitir al administrador registrar, modificar y desactivar las cuentas de los usuarios del sistema.

RF-04 El sistema debe gestionar la información de los entrenadores, incluyendo identificador único, nombre y especialidad, así como los programas de entrenamiento para los cuales están autorizados a generar o validar rutinas.

RF-05 El sistema debe gestionar la información de los clientes, incluyendo identificador único, nombre, edad y sede a la que pertenece.

RF-06 El sistema debe gestionar las sedes del gimnasio, de modo que cada cliente quede adscrito a una única sede y los reportes puedan agrupar resultados por sede.

RF-07 El sistema debe garantizar la separación de las funcionalidades de administrador, entrenador, jefe de sala y cliente, de modo que un usuario sólo acceda a las operaciones que le corresponden.

2.1.2. Programas de entrenamiento y grupos musculares

RF-08 El sistema debe gestionar los programas de entrenamiento, incluyendo identificador único, nombre, objetivo al que corresponde y la lista de grupos musculares a cubrir a lo largo del ciclo.

RF-09 El sistema debe permitir crear, modificar y eliminar los grupos musculares que componen el catálogo del dominio.

RF-10 El sistema debe permitir asignar a un programa de entrenamiento los grupos musculares que debe cubrir, y consultar dicha asignación.

2.1.3. Banco de ejercicios y su clasificación

RF-11 El sistema debe permitir a los entrenadores registrar nuevos ejercicios en el banco existente, capturando nombre, descripción y fecha de registro.

RF-12 El sistema debe permitir al entrenador clasificar los ejercicios descritos por él mismo según grupo muscular, tipo de ejercicio (fuerza, cardio o flexibilidad) y nivel de intensidad (bajo, medio o alto).

RF-13 El sistema debe permitir a todos los usuarios consultar el banco de ejercicios del sistema.

RF-14 El administrador debe poder identificar y prevenir la existencia de ejercicios duplicados en el banco, según el criterio de duplicidad establecido por el sistema.

RF-15 El sistema debe garantizar que la modificación de la clasificación de un ejercicio sólo pueda ser realizada por el entrenador que lo describió.

2.1.4. Autorización de entrenadores e inscripción de clientes

RF-16 El sistema debe permitir asignar a un entrenador los programas de entrenamiento sobre los que está autorizado a generar rutinas, a validarlas o a realizar ambas operaciones.

RF-17 El sistema debe permitir registrar la inscripción de un cliente en los programas de entrenamiento en los que participa, con su fecha de inscripción.

RF-18 El sistema debe impedir la generación de una rutina por parte de un entrenador sobre un programa para el que no se encuentre autorizado.

RF-19 El sistema debe permitir que un entrenador consulte las rutinas creadas por otros entrenadores cuando estos atiendan el mismo programa de entrenamiento.

2.1.5. Generación y composición de rutinas

RF-20 El sistema debe permitir a un entrenador generar rutinas de forma automática a partir de los ejercicios seleccionados.

RF-21 El sistema debe permitir establecer los criterios de generación de una rutina: nivel de intensidad objetivo, proporción de ejercicios por tipo, cobertura de los grupos musculares y cantidad total de ejercicios.

RF-22 El sistema debe almacenar, junto a la rutina generada, la parametrización utilizada durante su generación.

RF-23 El sistema debe registrar, para cada rutina generada, el programa de entrenamiento al que corresponde, el entrenador que la creó y la fecha de creación.

RF-24 El sistema debe registrar los ejercicios que componen cada rutina, indicando su orden dentro de la misma, el número de series, las repeticiones y el tiempo de descanso asociado.

RF-25 El sistema debe permitir al jefe de sala revisar una rutina generada, registrar la fecha de la revisión y las observaciones realizadas.

RF-26 El sistema debe permitir al jefe de sala aprobar una rutina o indicar la confección de una nueva bajo sus propios criterios.

RF-27 El sistema debe asociar cada rutina generada como resultado de una revisión con la revisión que le dio origen.

2.1.6. Asignación de rutinas y ejecución de sesiones

RF-28 El sistema debe permitir asignar rutinas a los clientes, y garantizar que no existan asignaciones duplicadas de una misma rutina a un mismo cliente.

RF-29 El sistema debe permitir que un cliente acceda a las rutinas que le han sido asignadas.

RF-30 El sistema debe almacenar cada sesión registrada ante una rutina ejecutada por un cliente, indicando fecha, hora de inicio, hora de fin, intensidad percibida y si la sesión fue completada.

RF-31 El sistema debe permitir a clientes y entrenadores consultar los reportes de progreso de los clientes en los programas de entrenamiento en los que están inscritos.

RF-32 El sistema debe permitir a un cliente solicitar de forma virtual el ajuste de una rutina que le ha sido asignada.

RF-33 El sistema debe permitir al cliente especificar el entrenador que se encargará de atender su solicitud de ajuste.

RF-34 El sistema debe registrar toda solicitud de ajuste, así como el resultado del ajuste manual realizado.

2.1.7. Consultas y reportes

RF-35 Obtener el listado de rutinas generadas automáticamente para un programa de entrenamiento específico, indicando el nombre del creador, la fecha de creación y los parámetros utilizados.

RF-36 Obtener los ejercicios más utilizados en las rutinas finales de un programa de entrenamiento, clasificados por nivel de intensidad y grupo muscular.

RF-37 Listar las rutinas que fueron validadas por un revisor determinado, indicando la fecha de validación y las observaciones hechas durante el proceso.

RF-38 Generar un reporte sobre el desempeño de los clientes en una rutina, clasificando los ejercicios por intensidad y comparando las tasas de finalización.

RF-39 Comparar las rutinas generadas para diferentes programas de entrenamiento, verificando la distribución de ejercicios por grupo muscular y nivel de intensidad y si los criterios de equilibrio fueron cumplidos.

RF-40 Determinar, para cada programa de entrenamiento, la correlación entre el nivel de intensidad de los ejercicios y el rendimiento promedio de los clientes.

RF-41 Identificar los 10 ejercicios con mayor tasa de abandono, indicando el entrenador que los creó y el programa de entrenamiento al que pertenecen.

RF-42 Comparar el desempeño de los clientes que solicitaron un ajuste de rutina con el promedio general de finalización de sus respectivas sedes, para los mismos programas de entrenamiento.

2.1.8. Presentación y exportación de la información

RF-43 El sistema debe presentar los resultados de las consultas y reportes mediante tablas y gráficos.

RF-44 El sistema debe permitir exportar a formato PDF la información mostrada, para todo tipo de usuario.

RF-45 El sistema debe permitir ordenar cada columna de los resultados de acuerdo con los intereses del usuario final.

2.2. Requerimientos no funcionales

Los requerimientos no funcionales (RNF) condicionan la solución sin describir operaciones concretas.

RNF-01 **Aplicación web y multiplataforma.** La solución debe implementarse como una aplicación web ejecutable en diversos sistemas operativos y navegadores.

RNF-02 **Interfaz amigable.** La interfaz de usuario debe ser amigable, intuitiva y de fácil uso.

RNF-03 **Disponibilidad del dato activo.** El sistema debe garantizar que la rutina activa de un cliente esté disponible de forma prácticamente inmediata durante la propia sesión de entrenamiento.

RNF-04 **Escalabilidad y concurrencia.** El rendimiento debe mantenerse con múltiples usuarios conectados de manera simultánea.

RNF-05 **Arquitectura desacoplada.** La aplicación debe utilizar una arquitectura que separe el front-end del back-end, facilitando su extensibilidad y mantenimiento.

RNF-06 **Patrones de diseño.** La solución debe implementar al menos dos patrones de diseño de software.

RNF-07 **Buenas prácticas de programación.** El código debe seguir buenas prácticas y contener documentación mediante comentarios estructurados.

RNF-08 **Pruebas unitarias.** Se deben implementar pruebas unitarias para el front-end y el back-end.

RNF-09 **Control de versiones.** El desarrollo debe gestionarse mediante un sistema de control de versiones, con commits frecuentes y atómicos que reflejen el aporte de cada integrante.

RNF-10 **Planificación CASE.** La planificación del proyecto debe realizarse utilizando alguna herramienta CASE.

RNF-11 **Persistencia de los datos.** Todas las operaciones relevantes del sistema deben quedar registradas y almacenadas de forma permanente, garantizando la trazabilidad histórica de la información.

RNF-12 **Exportación y ordenamiento.** La exportación a PDF y el ordenamiento dinámico de columnas deben estar disponibles para todo tipo de usuario.

2.3. Requerimientos informacionales

Los requerimientos informacionales (RI) describen la estructura de datos que la base de datos debe almacenar y organizar.

2.3.1. Gestión de usuarios, roles y sedes

RI-01 Almacenar el identificador único, nombre, correo electrónico, contraseña, rol y estado de cada usuario del sistema.

RI-02 Almacenar la especialidad de cada entrenador.

RI-03 Almacenar la edad de cada cliente y la sede a la que pertenece.

RI-04 Almacenar el identificador y nombre de cada sede del gimnasio.

2.3.2. Programas de entrenamiento y catálogos

RI-05 Almacenar el identificador único, nombre y objetivo de cada programa de entrenamiento.

RI-06 Almacenar los grupos musculares que cada programa de entrenamiento debe cubrir a lo largo del ciclo.

RI-07 Almacenar el catálogo de grupos musculares del dominio.

RI-08 Almacenar el catálogo de tipos de ejercicio (fuerza, cardio y flexibilidad).

RI-09 Almacenar el catálogo de niveles de intensidad (bajo, medio y alto).

2.3.3. Banco de ejercicios

RI-10 Almacenar el identificador, nombre, descripción y fecha de registro de cada ejercicio.

RI-11 Almacenar, para cada ejercicio, su tipo, su nivel de intensidad y su grupo muscular.

RI-12 Almacenar el entrenador que describió cada ejercicio.

RI-13 Mantener la información necesaria para identificar e impedir ejercicios duplicados conforme al criterio de duplicidad establecido.

2.3.4. Autorización e inscripción

RI-14 Almacenar, para cada relación de autorización entre un entrenador y un programa de entrenamiento, si puede generar rutinas, si puede validarlas, y el período de vigencia de la autorización.

RI-15 Almacenar la inscripción de cada cliente en los programas de entrenamiento en los que participa, con su fecha.

2.3.5. Generación y composición de rutinas

RI-16 Almacenar el identificador, la fecha de creación y el estado de cada rutina generada, así como el programa al que pertenece y el entrenador que la creó.

RI-17 Almacenar los parámetros utilizados en la generación de cada rutina: nivel de intensidad objetivo, proporción de ejercicios por tipo, cobertura de grupos musculares y cantidad total de ejercicios.

RI-18 Almacenar la composición de cada rutina, indicando para cada ejercicio su orden, el número de series, las repeticiones y el tiempo de descanso.

RI-19 Almacenar el estado de la rutina a efectos de revisión y validación, y mantener la referencia a la rutina que originó a una nueva generación.

2.3.6. Validación

RI-20 Almacenar para cada revisión de una rutina el jefe de sala que la realiza, la fecha de la revisión, las observaciones y la decisión adoptada.

RI-21 Almacenar los criterios que el jefe de sala establece cuando solicita la generación de una nueva rutina, manteniendo la relación entre la revisión y la rutina resultante.

2.3.7. Asignación y ejecución

RI-22 Almacenar las asignaciones de rutinas a clientes, identificando el cliente, la rutina asignada y las fechas de asignación y de finalización, cuando corresponda.

RI-23 Almacenar cada sesión registrada por un cliente sobre una rutina asignada, con su fecha, hora de inicio, hora de fin, intensidad percibida, si fue completada y las observaciones.

RI-24 Proporcionar la información necesaria para identificar y evitar asignaciones repetidas de una misma rutina a un mismo cliente.

2.3.8. Solicitudes de ajuste

RI-25 Almacenar para cada solicitud de ajuste el cliente que la realiza, la rutina sobre la que versa, el entrenador solicitado, la fecha de la solicitud, su motivo y su estado.

RI-26 Almacenar el entrenador que resuelve la solicitud, la fecha de respuesta, el resultado obtenido y la rutina resultante del ajuste.

RI-27 Mantener la relación entre la solicitud de ajuste, la rutina original y la rutina resultante.

2.3.9. Soporte de consultas y reportes

RI-28 Proporcionar la información necesaria para listar las rutinas generadas automáticamente para un programa de entrenamiento, incluyendo creador, fecha de creación y parámetros utilizados.

RI-29 Proporcionar la información necesaria para determinar los ejercicios más utilizados en las rutinas finales de un programa de entrenamiento, clasificándolos por nivel de intensidad y grupo muscular.

RI-30 Proporcionar la información necesaria para listar las rutinas validadas por un revisor determinado, incluyendo la fecha de validación y las observaciones realizadas.

RI-31 Proporcionar la información necesaria para generar reportes sobre el desempeño de los clientes en una rutina, clasificando los ejercicios por nivel de intensidad y comparando las tasas de finalización.

RI-32 Proporcionar la información necesaria para comparar las rutinas generadas para diferentes programas de entrenamiento, considerando la distribución de ejercicios por grupo muscular y nivel de intensidad y el cumplimiento de los criterios de equilibrio.

RI-33 Proporcionar, para cada programa de entrenamiento, el nivel de intensidad de los ejercicios, el rendimiento promedio medido como tasa de finalización y la correlación entre ambos indicadores.

RI-34 Proporcionar la información necesaria para identificar los 10 ejercicios con mayor tasa de abandono, indicando el entrenador que lo creó y el programa de entrenamiento al que pertenece.

RI-35 Proporcionar la información necesaria para comparar el desempeño de los clientes que solicitaron ajustes de rutina con el promedio general de finalización de sus respectivas sedes, para los mismos programas de entrenamiento.

3. Modelo Conceptual Relacional-Extendido (MERX)

El MERX representa el dominio del problema mediante conjuntos de entidades, interrelaciones con sus cardinalidades y generalizaciones. Se emplea notación de Chen: los rectángulos representan conjuntos de entidades y los rombos, interrelaciones; el hexágono *Type* que corona la arista de generalización señala la especialización de *User* en *Client* y *Trainer*; y los símbolos de doble línea (*Parameter* y su relación *Parameters - Routine*) representan una entidad y una relación débiles. Las cardinalidades se expresan en notación de Crow: en cada extremo de la arista se indica el número máximo de ocurrencias de la entidad del extremo contrario que pueden asociarse a una ocurrencia de la entidad de ese extremo. El valor mínimo se indica explícitamente cuando es distinto de uno, de modo que toda arista lleva una cardinalidad de la forma *mínimo,máximo*, que es la misma que se emplea en la sección 4.6.

Por un tema de limpieza visual, no colocamos los atributos en el MERX. El diagrama se limita a entidades, interrelaciones y cardinalidades. Los atributos de cada conjunto de entidades se listan en el esquema relacional de la sección 3.1, donde además se distinguen clave primaria y clave foránea; los atributos propios de cada interrelación se enumeran en las secciones 4 y 4.6.

El modelo completo se presenta en la figura 1.

Antes de presentar el diagrama conviene justificar tres decisiones de modelado que condicionan todo el esquema:

- El papel de *Room Chief* (Jefe de Sala) no se modela como una entidad especializada independiente de *Trainer*: las responsabilidades de revisión y validación son ejercidas por entrenadores, quedando su rol diferenciado funcionalmente a través del atributo *especialidad* y su asignación en la interrelación `VALIDATE`.
- El papel de administrador tampoco se modela como una entidad aparte: según el enunciado, el administrador no aporta atributos propios ni relaciones exclusivas, sino que supervisa la incorporación de ejercicios y las asignaciones de rutinas. Sus dos responsabilidades (evitar ejercicios duplicados e impedir la asignación doble) se implementan como restricciones de unicidad y de negocio sobre entidades que ya existen (U-12 y U-9). Por ello, `USER` mantiene la generalización hacia únicamente `CLIENT` y `TRAINER`.
- Los conceptos de tipo de ejercicio e intensidad no requieren entidades de catálogo externas en el MERX, sino que se incluyen directamente como atributos atómicos del conjunto de entidades `EXERCISE` (*tipo* e *intensidad*), manteniendo únicamente a `MUSCLE_GROUP` y `SITE` como entidades independientes asociadas.

3.1. Esquema relacional derivado del MERX

El paso del modelo conceptual al esquema relacional se apoya en cuatro reglas que se aplican de forma sistemática:

- La superentidad `USER` y sus dos especializaciones se materializan en relaciones distintas. La clave primaria de `USER` es a la vez clave primaria de `TRAINER` y `CLIENT`, de modo que la generalización se resuelve con claves foráneas idénticas y sin duplicar atributos.
- Toda interrelación de cardinalidad máxima N en ambos extremos se resuelve con una relación intermedia que incorpora los atributos propios de la interrelación, como es el caso de `AUTHORISES` o `CONTAINS`.
- Toda interrelación de cardinalidad máxima 1 se resuelve con una clave foránea en la relación del lado N: es el caso de `PARAMETERS_ROUTINE` y de `ADJUSTMENT_RESULT`.

- Los catálogos persistentes como entidades independientes (`SITE` y `MUSCLE_GROUP`) se referencian por clave foránea desde las entidades que los utilizan, mientras que clasificadores directos como tipo de ejercicio e intensidad se integran como atributos de dominio acotado.

El esquema resultante se presenta a continuación. Se subrayan las claves primarias y se marcan en negrita las claves foráneas; un atributo subrayado y en negrita es a la vez clave primaria y foránea.

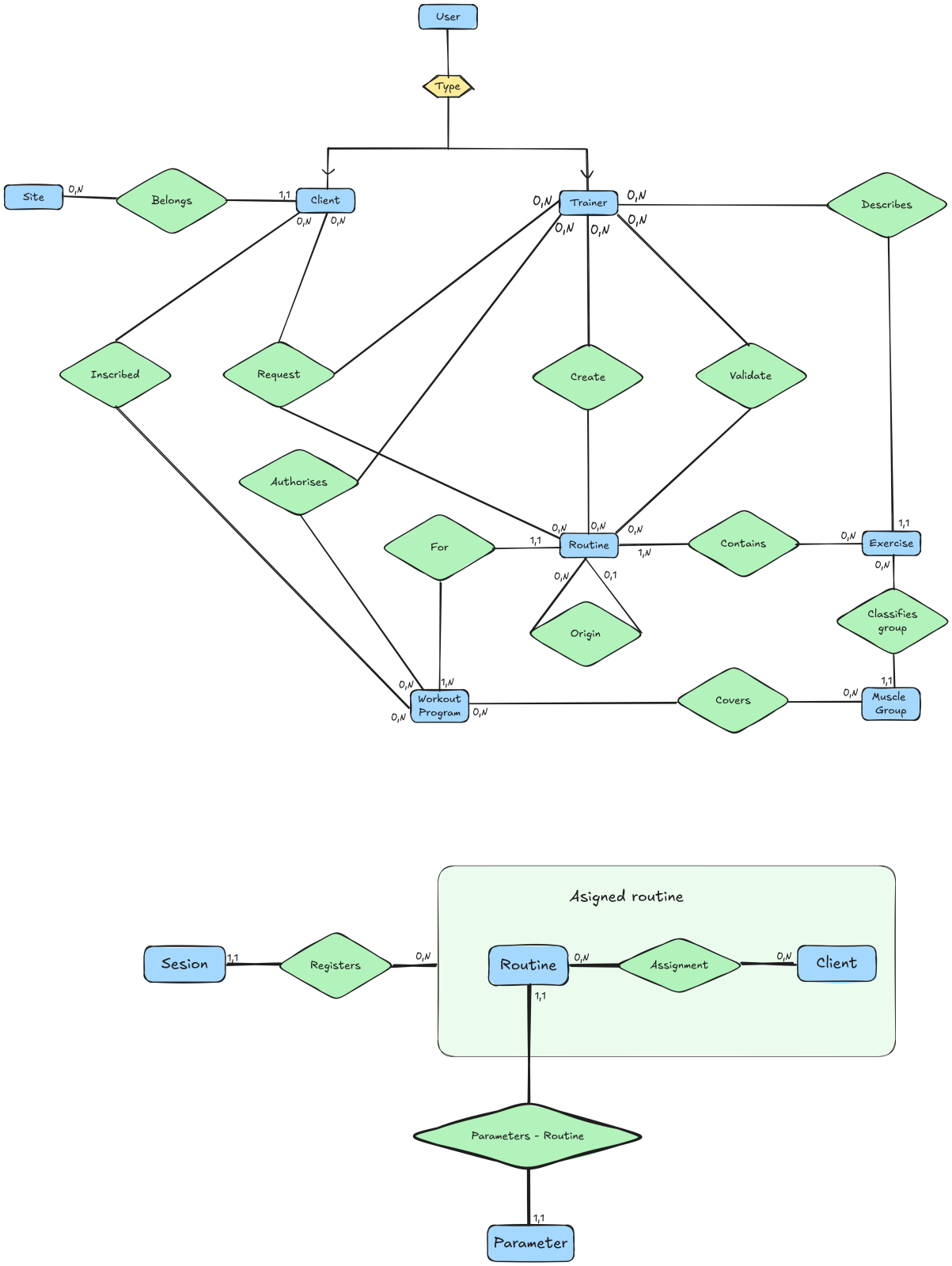


Figura 1: Modelo Conceptual Relacional-Extendido (MERX) del sistema de generación automática de planes de entrenamiento. Rectángulos: conjuntos de entidades; rombos: interrelaciones; hexágono *Type*: generalización de *User* en *Client* y *Trainer*; doble línea: entidad y relación débiles; y el recuadro *Assigned routine* agrupa la interrelación *Assignment* con sus dos participantes. En cada extremo de cada arista se indica la cardinalidad en notación *mínimo,máximo*. Por un tema de limpieza visual no se muestran los atributos: sus listados figuran en la sección 3.1.

Cuadro 1: Correspondencia entre las etiquetas de la figura 1 y las relaciones del esquema de la sección 3.1.

Etiqueta en la figura	**Relación del esquema**
Conjuntos de entidades	
User	R1 `USER` (superentidad)
Client	R3 `CLIENT` (especialización)
Trainer	R2 `TRAINER` (especialización)
Site	R4 `SITE` (catálogo)
Workout Program	R5 `WORKOUT_PROGRAM`
Muscle group	R7 `MUSCLE_GROUP` (catálogo)
Exercise	R8 `EXERCISE`
Routine	R11 `ROUTINE`
Sesion	R16 `SESION`
Parameter	R12 `PARAMETERS_ROUTINE` (entidad débil)
Assigned routine	R15 `ASSIGNMENT` (recuadro que agrupa la interrelación)
Interrelaciones	
Type (generalización)	`USER.rol` más R2 y R3
Belongs	R4 clave foránea R3.`idSite`
Inscribed	R10 `INSCRIBED`
Authorises	R9 `AUTHORISES`
Request (ternaria)	R17 `REQUEST`
Create	R2 clave foránea R11.`idTrainerCreator`
Validate	R14 `VALIDATE`
Describes	R2 clave foránea R8.`idTrainerCreator`
For	R5 clave foránea R11.`idWorkoutProgram`
Contains	R13 `CONTAINS`
Origin	R11 clave foránea R11.`idRoutineOrigin`
Classifies group	R7 clave foránea R8.`idMuscleGroup`
Covers	R6 `COVERS`
Assignment	R15 `ASSIGNMENT`
Registers	R15 clave foránea R16.`idAssignment`
Parameters - Routine	R12 `PARAMETERS_ROUTINE` (relación débil)

Esquema relacional

R1 `USER`(idUser, nombre, correo, contrasena, rol, activo)

R2 `TRAINER`(**idTrainer**, especialidad)

R3 `CLIENT`(**idClient**, edad, **idSite**)

R4 `SITE`(idSite, nombre)

R5 `WORKOUT_PROGRAM`(idWorkoutProgram, nombre, objetivo)

R6 `COVERS`(**idWorkoutProgram**, **idMuscleGroup**)

R7 `MUSCLE_GROUP`(idMuscleGroup, nombre)

R8 `EXERCISE`(idExercise, nombre, descripcion, fechaRegistro, tipo, intensidad, **idMuscleGroup**, **idTrainerCreator**)

R9 AUTHORISES(**idTrainer**, **idWorkoutProgram**, puedeGenerar, puedeValidar, fechaInicio, fechaFin)

R10 INSCRIBED(**idClient**, **idWorkoutProgram**, fechaInscripcion)

R11 ROUTINE(idRoutine, fechaCreacion, estado, refEstructura, **idWorkoutProgram**, **idTrainerCreator**, **idRoutineOrigin**)

R12 PARAMETERS_ROUTINE(idParameters, **idRoutine**, intensidadObjetiva, cantidadTotalEjercicios, proporcionFuerza, proporcionCardio, proporcionFlexibilidad)

R13 CONTAINS(**idRoutine**, **idExercise**, orden, series, repeticiones, descansoSegundos)

R14 VALIDATE(idValidation, **idRoutine**, **idTrainerEvaluador**, fechaValidacion, decision, observaciones)

R15 ASSIGNMENT(idAssignment, **idRoutine**, **idClient**, fechaAsignacion, fechaFin)

R16 SESION(idSesion, **idAssignment**, fechaSesion, horaInicio, horaFin, intensidadPercibida, completada, observaciones)

R17 REQUEST(idRequest, **idClient**, **idRoutine**, **idTrainer**, fechaSolicitud, motivo, estado)

R18 ADJUSTMENT_RESULT(**idRequest**, fechaRespuesta, descripcionAjuste, **idRoutineResultante**, **idTrainerResolutor**)

El esquema declara las claves foráneas detalladas a continuación, que constituyen la base de las restricciones de integridad referencial. La primera columna indica la relación que contiene la clave foránea y la segunda, la relación referenciada.

Rel.	Claves foráneas y relación referenciada
R2	idTrainer → R1(idUser)
R3	idClient → R1(idUser); idSite → R4(idSite)
R6	idWorkoutProgram → R5(idWorkoutProgram); idMuscleGroup → R7(idMuscleGroup)
R8	idMuscleGroup → R7(idMuscleGroup); idTrainerCreator → R2(idTrainer)
R9	idTrainer → R2(idTrainer); idWorkoutProgram → R5(idWorkoutProgram)
R10	idClient → R3(idClient); idWorkoutProgram → R5(idWorkoutProgram)
R11	idWorkoutProgram → R5(idWorkoutProgram); idTrainerCreator → R2(idTrainer); idRoutineOrigin → R11(idRoutine)
R12	idRoutine → R11(idRoutine)
R13	idRoutine → R11(idRoutine); idExercise → R8(idExercise)
R14	idRoutine → R11(idRoutine); idTrainerEvaluador → R2(idTrainer)
R15	idRoutine → R11(idRoutine); idClient → R3(idClient)
R16	idAssignment → R15(idAssignment)
R17	idClient → R3(idClient); idRoutine → R11(idRoutine); idTrainer → R2(idTrainer)
R18	idRequest → R17(idRequest); idRoutineResultante → R11(idRoutine); idTrainerResolutor → R2(idTrainer)

Dos decisiones de este esquema merecen comentario, porque son las que evitan que aparezcan anomalías más adelante:

- La interrelación *CLIENTE registra SESION* que se dibujó en el MERX no se materializa como clave foránea propia en SESION. El cliente que ejecuta una sesión se alcanza a través de SESION → ASSIGNMENT → idClient. Guardarlo también en SESION introduciría la dependencia transitiva *idSesion* → *idAssignment* → *idClient*, que es justamente lo que impide estar en tercera forma normal. En consecuencia, toda sesión queda obligatoriamente

ligada a una asignación y, por extensión, a un cliente y a una rutina.

- `ASSIGNMENT` lleva su propio identificador (`idAssignment`) aunque la combinación de rutina y cliente ya sea única. Ese identificador es lo que permite registrar las sesiones contra una asignación concreta y lo que hace posible la restricción U-9 de no repetir la misma rutina al mismo cliente.

3.2. Estructura variable de bloques de la rutina

El enunciado del problema advierte que la descripción interna de una rutina puede constar de cantidades variables de calentamientos, circuitos, series y descansos, con elementos anidados, y que dicha descripción no se ajusta a un esquema fijo y común para todas las rutinas.

Por tal motivo, y siguiendo de forma estricta la indicación del enunciado, esa estructura no se modela junto al resto de las tablas de la solución. El atributo `refEstructura` de la entidad `ROUTINE` funciona como un identificador opaco hacia un almacén de datos independiente, orientado a documentos o a jerarquías anidadas, donde cada rutina conserva su propia estructura. La sección 4.9 detalla el tratamiento de esta decisión.

4. Especificación de las restricciones de integridad establecidas y/o detectadas

Las restricciones de integridad constituyen el conjunto de condiciones que debe satisfacer cualquier ocurrencia válida de la base de datos. Se organizan en cinco familias, siguiendo la clasificación clásica: integridad de entidad, de dominio, referencial, semántica (o de negocio) y temporal. Para cada restricción se indica el mecanismo concreto mediante el cual se implantará.

4.1. Integridad de entidad

Toda entidad del modelo debe poder identificarse de manera inequívoca. Por tanto:

- Cada programa de entrenamiento posee un identificador único.
- Cada entrenador posee un identificador único.
- Cada cliente posee un identificador único.
- Cada sede y grupo muscular poseen un identificador único (los tipos de ejercicio y niveles de intensidad se gestionan como atributos de dominio acotado en `EXERCISE`).
- Cada ejercicio, rutina, rutina parametrizada, validación, asignación, sesión, solicitud de ajuste y resultado de ajuste poseen una clave primaria que permite distinguir inequívocamente cada una de sus ocurrencias.
- Los atributos que conforman las claves primarias no admiten valores nulos.
- No pueden existir dos ocurrencias con el mismo valor de clave primaria.

4.2. Integridad de dominio

Los atributos de las distintas entidades aceptan únicamente valores pertenecientes a su dominio correspondiente.

- El rol de un usuario sólo puede tomar los valores *administrador*, *entrenador* y *cliente*.
- El nivel de intensidad sólo puede tomar los valores *bajo*, *medio* o *alto*. Este dominio se aplica tanto a la clasificación de los ejercicios como a la intensidad objetivo de una rutina y a la intensidad percibida en una sesión.
- El tipo de ejercicio sólo puede tomar los valores *fuerza*, *cardio* o *flexibilidad*.
- La edad del cliente debe ser un número entero positivo.
- Las fechas asociadas a la creación, validación, ejecución, asignación y modificación de una rutina deben corresponder a valores de fecha válidos.
- Los atributos obligatorios definidos en el modelo, como nombres e identificadores, no pueden tomar valores nulos.
- En los atributos que representen cantidades, porcentajes, repeticiones, tiempos u otros valores numéricos no se admitirán valores negativos.
- La cantidad total de ejercicios de una rutina debe ser un entero estrictamente mayor que cero.
- El orden de un ejercicio dentro de una rutina debe ser un entero estrictamente positivo.
- Las proporciones por tipo de ejercicio de la parametrización deben ser valores no negativos cuya suma sea la unidad.

4.3. Integridad referencial

Toda clave foránea debe referenciar una ocurrencia existente de la entidad relacionada. Las principales restricciones son las siguientes:

- Todo programa asignado a un cliente mediante `INSCRIBED` debe existir previamente en `WORKOUT_PROGRAM`.
- Todo programa para el cual un entrenador se encuentre autorizado debe existir en `WORKOUT_PROGRAM`.
- Todo ejercicio asociado a una rutina debe corresponder a un ejercicio registrado previamente en el banco de ejercicios (`EXERCISE`).
- Toda rutina debe estar asociada al programa de entrenamiento para el cual fue creada y al entrenador que la creó.
- El entrenador registrado como creador de una rutina debe existir en `TRAINER`.
- El entrenador que valida una rutina debe existir y estar autorizado para realizar dicha acción sobre el programa de entrenamiento correspondiente.
- Toda rutina asignada a un cliente debe hacer referencia a un cliente, una rutina y un programa de entrenamiento existentes.
- Toda sesión de entrenamiento registrada debe estar vinculada con una asignación previamente existente (`ASSIGNMENT`); no puede haber sesiones sobre combinaciones cliente-rutina que no posean una asignación válida.
- Toda solicitud de ajuste debe corresponder a un cliente, a un programa y al entrenador seleccionado para atenderla (`REQUEST`).
- Todo resultado de ajuste (`ADJUSTMENT_RESULT`) debe corresponder a una solicitud existente.

- No debe eliminarse una ocurrencia que resulte necesaria para mantener registros históricos asociados, salvo que se establezca explícitamente una política de eliminación o actualización en cascada que no provoque pérdida de información relevante.

4.4. Integridad de unicidad

Se identifican doce restricciones de unicidad, que se numeran como U-n porque la valoración de la sección 5 las cita al justificar las claves candidatas.

U-1 El correo electrónico de un usuario es único: no pueden existir dos usuarios con el mismo correo. Es la causa de que *correo* sea clave candidata alternativa de `USER`.

U-2 El nombre de un programa de entrenamiento es único, por lo que *nombre* es clave candidata alternativa de `WORKOUT_PROGRAM`.

U-3 Los nombres de los catálogos (grupo muscular y sede) son únicos dentro de su respectivo catálogo, o bien por restricción de unicidad aplicada, lo que convierte a *nombre* en clave candidata alternativa de `MUSCLE_GROUP` y `SITE`.

U-4 El par (`idClient`, `idWorkoutProgram`) es único en `INSCRIBED`: un cliente no puede inscribirse dos veces en el mismo programa.

U-5 El par (`idTrainer`, `idWorkoutProgram`) es único en `AUTHORISES`: un entrenador no puede tener dos autorizaciones simultáneas para el mismo programa.

U-6 El par (`idRoutine`, `idExercise`) es único en `CONTAINS`: un mismo ejercicio no puede repetirse dentro de una misma rutina.

U-7 El par (`idRoutine`, `orden`) es único en `CONTAINS`: no puede haber dos ejercicios en la misma posición de la rutina. Junto con U-6, da a `CONTAINS` dos claves candidatas distintas.

U-8 `idRoutine` es único en `PARAMETERS_ROUTINE`: cada rutina tiene exactamente una parametrización, o bien almacenada de forma asociada, lo que hace de `idRoutine` clave candidata alternativa de esa relación.

U-9 El par (`idRoutine`, `idClient`) es único en `ASSIGNMENT`: no se puede asignar dos veces la misma rutina al mismo cliente. Es la que impide la asignación doble que el enunciado atribuye al administrador.

U-10 El par (`idAssignment`, `fechaSesion`) es único en `SESION`: un cliente no puede registrar dos sesiones el mismo día sobre la misma asignación.

U-11 `idRequest` es único en `ADJUSTMENT_RESULT`: cada solicitud admite a lo sumo un resultado, en correspondencia con la cardinalidad $0,1$ de la sección 4.6.

U-12 No pueden existir dos ejercicios duplicados en el banco de ejercicios. El criterio de duplicidad se define como la coincidencia del nombre del ejercicio junto con su grupo muscular, de modo que la combinación (`nombre`, `idMuscleGroup`) es única en `EXERCISE` y constituye su segunda clave candidata.

4.5. Nulabilidad

- Todos los identificadores y claves primarias son obligatorios (`NOT NULL`).
- Son obligatorios el nombre de toda entidad, el nombre del objetivo de un programa y la especialidad de un entrenador.

- Son obligatorios la edad y la sede de un cliente.
- Son obligatorios el tipo, el nivel de intensidad, el grupo muscular y el entrenador creador de un ejercicio.
- Es obligatoria la fecha de inscripción, de creación de rutina, de validación, de asignación, de sesión, de solicitud y de respuesta.
- Son obligatorios el motivo de la solicitud de ajuste, la descripción del ajuste realizado y la decisión adoptada en una validación.
- Son opcionales (anulables) la descripción de un ejercicio, la fecha de fin de una autorización, la fecha de fin de una asignación, las observaciones de una validación, la intensidad percibida de una sesión y las observaciones de una sesión.
- Es opcional la referencia a la rutina origen en `ROUTINE`: sólo existe cuando la rutina fue generada como resultado de una revisión.

4.6. Restricciones de cardinalidad por interrelación

Se emplea la notación mínimo,máximo en la forma $(0,1)$, $(1,1)$, $(0,N)$, $(1,N)$ y (N,N). El valor mínimo indica si la participación es obligatoria (1) u opcional (0); el máximo, si puede ocurrir más de una vez. Las dos especializaciones de `USER` se expresan en la forma $(0,1)/(1,1)$ porque la pertenencia al subtipo es opcional para el usuario pero obligatoria para el subtipo.

4.6.1 User — Trainer Un usuario puede ser entrenador entre cero y una vez $(0,1)$; un entrenador es exactamente un usuario $(1,1)$.

4.6.2 User — Client Un usuario puede ser cliente entre cero y una vez $(0,1)$; un cliente es exactamente un usuario $(1,1)$.

4.6.3 Client — Site Cada cliente pertenece a exactamente una sede $(1,1)$; cada sede agrupa entre cero y muchos clientes $(0,N)$.

4.6.4 Trainer — AUTHORISES — Workout Program Un entrenador puede estar autorizado en entre cero y muchos programas $(0,N)$; un programa puede tener entre cero y muchos entrenadores autorizados $(0,N)$. La interrelación aporta los atributos `puedeGenerar`, `puedeValidar`, `fechaInicio` y `fechaFin`.

4.6.5 Client — INSCRIBED — Workout Program Un cliente puede estar inscrito en entre cero y muchos programas $(0,N)$; un programa puede tener entre cero y muchos clientes inscritos $(0,N)$. La interrelación aporta el atributo `fechaInscripcion`.

4.6.6 Trainer — Describes — Exercise Un entrenador puede describir entre cero y muchos ejercicios $(0,N)$; cada ejercicio es descrito por exactamente un entrenador $(1,1)$.

4.6.7 Exercise — Classifies group — Muscle group Cada ejercicio pertenece a exactamente un grupo muscular $(1,1)$; cada grupo muscular clasifica entre cero y muchos ejercicios $(0,N)$.

4.6.8 Workout Program — Covers — Muscle group Un programa cubre entre cero y muchos grupos musculares $(0,N)$; cada grupo muscular puede ser cubierto por entre cero y muchos programas $(0,N)$.

4.6.9 Trainer — Creates — Routine Un entrenador genera entre cero y muchas rutinas $(0, N)$; cada rutina es generada por exactamente un entrenador $(1, 1)$.

4.6.10 Routine — Parameters - Routine — Parameter Cada rutina tiene exactamente una parametrización $(1, 1)$; cada parametrización corresponde a exactamente una rutina $(1, 1)$.

4.6.11 Routine — Contains — Exercise Una rutina contiene entre uno y muchos ejercicios $(1, N)$; un ejercicio puede pertenecer a entre cero y muchas rutinas $(0, N)$. La interrelación aporta los atributos `orden`, `series`, `repeticiones` y `descansoSegundos`.

4.6.12 Trainer — Validate — Routine Un entrenador evaluador revisa entre cero y muchas rutinas $(0, N)$; cada rutina es revisada entre una y muchas veces $(1, N)$, pues puede ser reenviada tras un rechazo. La interrelación aporta `fechaValidacion`, `decision` y `observaciones`.

4.6.13 Routine — Assignment — Client Una rutina se asigna entre cero y muchos clientes $(0, N)$; un cliente recibe entre cero y muchas rutinas $(0, N)$. La interrelación aporta `fechaAsignacion` y `fechaFin`.

4.6.14 Assignment — Registers — Sesion Una asignación admite entre cero y muchas sesiones $(0, N)$; cada sesión corresponde a exactamente una asignación $(1, 1)$.

4.6.15 Client — Request — Routine — Trainer Es una interrelación ternaria: un cliente solicita el ajuste de una rutina indicando el entrenador que lo atenderá. Un cliente formula entre cero y muchas solicitudes $(0, N)$; un entrenador atiende entre cero y muchas solicitudes $(0, N)$; una rutina puede ser objeto de entre cero y muchas solicitudes $(0, N)$. La interrelación aporta `fechaSolicitud`, `motivo` y `estado`.

4.6.16 Request — Resuelve — Adjustment Una solicitud genera entre cero y un resultado de ajuste $(0, 1)$; cada resultado corresponde a exactamente una solicitud $(1, 1)$.

Las interrelaciones con atributos propios o con cardinalidad máxima N en ambos extremos se materializan como las relaciones asociativas `AUTHORISES`, `INSCRIBED`, `COVERS`, `CONTAINS`, `VALIDATE`, `ASSIGNMENT` y `REQUEST` del esquema de la sección 3.1. La interrelación ternaria *Request* se representa por `REQUEST`, que reagrupa sus tres participantes en una sola relación con clave primaria propia.

4.7. Restricciones semánticas y de negocio

Se identificaron reglas propias del funcionamiento del sistema que deben preservarse con independencia de que no puedan expresarse únicamente mediante claves primarias y foráneas.

R1. **No duplicidad de ejercicios.** El sistema debe impedir la existencia de ejercicios duplicados en el banco de ejercicios. Antes de incorporar un nuevo ejercicio debe comprobarse que no exista previamente otro considerado equivalente según el criterio de duplicidad establecido.

R2. **Clasificación por el creador.** Un entrenador puede clasificar por grupo muscular, tipo e intensidad aquellos ejercicios que hayan sido descritos por él. Por consiguiente, la modificación de dicha clasificación debe realizarse únicamente sobre ejercicios para los cuales el entrenador posea la relación de autoría.

R3. **Autorización de entrenadores.** Un entrenador sólo puede generar o validar rutinas pertenecientes a programas de entrenamiento para los cuales se encuentre autorizado.

Antes de registrar una generación o una validación debe verificarse la existencia de la correspondiente relación de autorización entre el entrenador y el programa.

R4. **Acceso a rutinas de otros entrenadores.** Un entrenador sólo puede consultar las rutinas creadas por otros entrenadores cuando estos atiendan el mismo programa de entrenamiento. Esta regla se comprueba a partir de la relación entre entrenadores y programas.

R5. **No duplicidad de asignaciones de rutinas.** Debe impedirse la asignación doble de una misma rutina a un cliente cuando represente la misma asignación activa dentro del programa correspondiente.

R6. **Inscripción previa del cliente.** Una rutina perteneciente a un determinado programa sólo podrá asignarse a un cliente que se encuentre inscrito en dicho programa de entrenamiento.

R7. **Registro de sesiones sobre rutinas asignadas.** Una sesión de entrenamiento únicamente puede registrarse para una rutina que haya sido previamente asignada al cliente que realiza dicha sesión. No deben existir sesiones asociadas a combinaciones cliente-rutina que no posean una asignación válida.

R8. **Consistencia de la generación automática.** Cada rutina generada automáticamente debe conservar los parámetros utilizados durante su generación, incluidos la proporción de ejercicios por tipo, la cobertura de grupos musculares y la cantidad total de ejercicios.

R9. **Coherencia de la cobertura muscular.** Los grupos musculares incluidos en una rutina deben ser coherentes con los grupos musculares que deben cubrirse en el programa correspondiente. Esta información permite comprobar además, posteriormente, si se cumplieron los criterios de equilibrio establecidos para la generación.

R10. **Aprobación de las rutinas.** La aprobación de una rutina debe ser realizada por un entrenador evaluador autorizado sobre el programa en cuestión. Cada aprobación debe conservar la fecha en que fue realizada y las observaciones producidas durante el proceso; cuando la decisión sea desfavorable, las observaciones son obligatorias.

R11. **Regeneración bajo criterios del revisor.** Cuando el entrenador evaluador indique la confección de otra rutina, la rutina generada como resultado debe conservar la referencia a la revisión que le dio origen, y la rutina anterior debe quedar marcada como sustituida.

R12. **Solicitudes de ajuste.** En una solicitud de ajuste el cliente debe especificar el entrenador que realizará el ajuste. Su resultado debe conservarse de forma relacionada con la solicitud que lo originó, con el fin de mantener la trazabilidad del proceso. Un cliente no puede formular una nueva solicitud de ajuste sobre la misma rutina mientras exista una solicitud pendiente para ella.

R13. **Identificación de las entidades asociativas.** Las entidades `AUTHORISES`, `INSCRIBED` y `CONTAINS` no admiten más de una ocurrencia para una misma combinación de participantes, garantizando la no duplicación de las relaciones que representan.

4.8. Restricciones temporales

Los distintos eventos relacionados con una rutina deben respetar un orden temporal coherente:

- Una rutina no puede ser validada antes de haber sido creada.
- Una sesión no puede corresponder a una fecha anterior a la fecha de asignación de la rutina al cliente.

- El resultado de una solicitud de ajuste no puede registrarse antes de la propia solicitud.
- La fecha de fin de una autorización no puede ser anterior a su fecha de inicio.
- La fecha de fin de una asignación no puede ser anterior a su fecha de asignación.
- Una rutina generada como resultado de una revisión no puede tener fecha de creación anterior a la de la revisión que la originó.
- La hora de fin de una sesión no puede ser anterior a su hora de inicio.

4.9. Tratamiento de la estructura variable de bloques

La estructura interna de una rutina puede estar formada por cantidades variables de calentamientos, circuitos, series, descansos y elementos anidados. Debido a su naturaleza variable, y siguiendo la decisión del equipo de unificar toda la persistencia bajo un único SGBD relacional, esta información se gestiona directamente en **PostgreSQL**.

La decisión se materializa de la siguiente manera:

- La entidad `ROUTINE` incorpora el atributo `refEstructura` empleando el tipo de dato nativo `JSONB` de PostgreSQL, el cual almacena de forma estructurada, indexable y flexible la jerarquía completa de bloques de la rutina.
- Cada rutina conserva así su propia estructura anidada (calentamientos, circuitos y series) sin verse obligada a ajustarse a un esquema relacional rígido de columnas fijas que rompería la normalización.
- El uso de `JSONB` en PostgreSQL permite realizar consultas y extracciones eficientes sobre los datos internos de la rutina utilizando operadores nativos, manteniendo al mismo tiempo la integridad transaccional, las claves foráneas y las restricciones en una única base de datos unificada.
- No deben existir estructuras huérfanas: la integridad referencial y de asociación lógica de la rutina se rige por las restricciones del núcleo relacional.

Esta arquitectura unificada simplifica la implementación del sistema al eliminar la necesidad de integrar un motor documental externo (como MongoDB). Asimismo, potencia el rendimiento y facilita el cumplimiento del requerimiento no funcional de disponibilidad inmediata del dato activo mediante el uso de índices GIN sobre las columnas `JSONB` del motor relacional.

4.10. Mecanismos de implantación de las restricciones

No todas las restricciones descritas pueden implementarse mediante el mismo mecanismo. Según su naturaleza, se utilizarán:

- `PRIMARY KEY`, para garantizar la identificación única de las entidades.
- `FOREIGN KEY`, para garantizar la integridad referencial.
- `UNIQUE`, para impedir valores o combinaciones de valores duplicados cuando corresponda.
- `NOT NULL`, para los atributos cuya presencia sea obligatoria.
- `CHECK`, para restringir dominios y validar condiciones simples sobre los atributos (incluyendo validaciones sobre la estructura JSONB).

- Restricciones implementadas mediante la lógica del gestor de base de datos (disparadores, *triggers*) o de la aplicación, para aquellas reglas que involucren varias entidades o requieran comprobaciones más complejas, como la no duplicidad de ejercicios, la verificación de la autorización del entrenador y el cumplimiento de la cobertura muscular.

5. Valoración de la corrección del diseño de la base de datos

5.1. Objetivo y metodología

Esta sección valora el diseño obtenido a partir del MERX. Se considera que un diseño es correcto cuando cumple cinco condiciones:

1. No presenta redundancia que produzca anomalías.
2. Todas las relaciones están en tercera forma normal (3FN) o superior.
3. La descomposición preserva las dependencias funcionales (PPDF).
4. La descomposición tiene la propiedad de join sin pérdida de información (PLJ).
5. Cubre los requerimientos funcionales e informacionales del sistema.

La valoración se realizó sobre el esquema relacional de la sección 3.1, compuesto por 18 relaciones integradas completamente en PostgreSQL (incorporando el atributo `refEstructura` de tipo `JSONB` en la entidad `ROUTINE`), siguiendo el procedimiento clásico de diseño relacional:

1. Extracción del universo de atributos U y del conjunto de dependencias funcionales F a partir del MERX y de las restricciones de la sección 4.
2. Cálculo del cubrimiento minimal $F_{\text{mín}}$.
3. Identificación de las claves candidatas y elección de la clave primaria.
4. Verificación de la 1FN, la 2FN y la 3FN relación por relación.
5. Comprobación de la PLJ con el Lema de Ullman y, en un caso, con el algoritmo de la tabla.
6. Comprobación de la PPDF con el algoritmo de la proyección.
7. Análisis de anomalías y de correspondencia con los requerimientos.

5.2. Universo de atributos y dependencias funcionales

El universo de atributos U es la unión de los atributos de las 18 relaciones. Para no repetir el esquema, se listan aquí las dependencias funcionales F, que se han obtenido aplicando tres reglas al MERX:

1. Por cada conjunto de entidades con atributos $X \subseteq U$ y clave K se añade $K \to X$.
2. Por cada conjunto de interrelaciones con atributos propios X y clave K se añade $K \to X$.
3. Por cada cardinalidad máxima 1, se añade la dependencia que permite recorrer la clave foránea: si R_j referencia a R_i por su clave K, se añade $K \to R_i \setminus K$ en la relación R_j.

El conjunto F queda formado por las dependencias que se resumen por relación en el Cuadro 2. Cada relación aporta además la dependencia de su clave foránea hacia la clave primaria que referencia, ya descrita en la sección 3.1.

5.3. Cubrimiento minimal

G es un cubrimiento minimal de F si se cumplen tres condiciones: $G^+ \equiv F^+$, G no contiene atributos redundantes en el determinante y G no contiene dependencias redundantes. El algoritmo aplicado fue el estándar: descomponer cada dependencia para que su lado derecho sea un único atributo, eliminar atributos redundantes del lado izquierdo comprobando las clausuras X^+, y eliminar las dependencias redundantes comprobando $A \in X^+_{G\setminus\{X\to A\}}$.

La descomposición del lado derecho no cambia nada en F, porque las dependencias se declararon con el lado derecho ya descompuesto en atributos simples, salvo en las relaciones con claves candidatas alternativas, donde el algoritmo elimina dependencias:

- En R1, al ser *correo* clave candidata, las cuatro dependencias con *correo* en el lado izquierdo siguen siendo necesarias, pero tres de las cinco con *idUser* son redundantes: *nombre*, *contrasena* y *rol* se obtienen encadenando $idUser \to correo$ con $correo \to$ ese atributo.
- En R4, R5 y R7, los catálogos o programas con nombre único siguen el mismo patrón: se conserva la ida por la clave artificial y la vuelta por el nombre, eliminando las dependencias del nombre hacia los atributos no clave.
- En R12, la cardinalidad 1 : 1 hace que $idRoutine \to idParameters$ e $idParameters \to idRoutine$ sean ambas claves candidatas. Las diez dependencias restantes, que apuntan a atributos no clave, son redundantes y se eliminan.
- En R13, R15 y R16 ocurre lo mismo con las dependencias hacia atributos no clave por existir dos claves candidatas.

El cubrimiento minimal resultante $F_{\text{mín}}$ queda distribuido como muestra el Cuadro 3.

Cuadro 2: Dependencias funcionales F por relación.

Rel.	**Dependencias funcionales de F**
R1	idUser → nombre, correo, contrasena, rol, activo; correo → nombre, contrasena, rol, activo
R2	idTrainer → especialidad
R3	idClient → edad, idSite
R4	idSite → nombre; nombre → idSite
R5	idWorkoutProgram → nombre, objetivo; nombre → idWorkoutProgram, objetivo
R6	(ninguna no trivial; todos los atributos forman la clave)
R7	idMuscleGroup → nombre; nombre → idMuscleGroup
R8	idExercise → nombre, descripcion, fechaRegistro, tipo, intensidad, idMuscleGroup, idTrainerCreator; (nombre, idMuscleGroup) → descripcion, fechaRegistro, tipo, intensidad, idTrainerCreator
R9	(idTrainer, idWorkoutProgram) → puedeGenerar, puedeValidar, fechaInicio, fechaFin
R10	(idClient, idWorkoutProgram) → fechaInscripcion
R11	idRoutine → fechaCreacion, estado, refEstructura, idWorkoutProgram, idTrainerCreator, idRoutineOrigen
R12	idParameters → idRoutine, intensidadObjetiva, cantidadTotalEjercicios, proporcionFuerza, proporcionCardio, proporcionFlexibilidad; idRoutine → idParameters, intensidadObjetiva, cantidadTotalEjercicios, proporcionFuerza, proporcionCardio, proporcionFlexibilidad

Rel.	Dependencias funcionales de F
R13	(idRoutine, idExercise) → orden, series, repeticiones, descansoSegundos; (idRoutine, orden) → idExercise, series, repeticiones, descansoSegundos
R14	idValidation → idRoutine, idTrainerEvaluador, fechaValidacion, decision, observaciones
R15	idAssignment → idRoutine, idClient, fechaAsignacion, fechaFin; (idRoutine, idClient) → idAssignment, fechaAsignacion, fechaFin
R16	idSesion → idAssignment, fechaSesion, horaInicio, horaFin, intensidadPercibida, completada, observaciones; (idAssignment, fechaSesion) → idSesion, horaInicio, horaFin, intensidadPercibida, completada, observaciones
R17	idRequest → idClient, idRoutine, idTrainer, fechaSolicitud, motivo, estado
R18	idRequest → fechaRespuesta, descripcionAjuste, idRoutineResultante, idTrainerResolutor

Cuadro 3: Cubrimiento minimal $F_{\text{mín}}$ por relación.

Rel.	DF	Dependencias de $F_{\text{mín}}$
R1	5	idUser → correo; correo → nombre, contrasena, rol, activo
R2	1	idTrainer → especialidad
R3	2	idClient → edad, idSite
R4	2	idSite → nombre; nombre → idSite
R5	2	idWorkoutProgram → nombre; nombre → idWorkoutProgram
R6	0	—
R7	2	idMuscleGroup → nombre; nombre → idMuscleGroup
R8	7	idExercise → nombre, idMuscleGroup; (nombre, idMuscleGroup) → descripcion, fechaRegistro, tipo, intensidad, idTrainerCreator
R9	4	(idTrainer, idWorkoutProgram) → puedeGenerar, puedeValidar, fechaInicio, fechaFin
R10	1	(idClient, idWorkoutProgram) → fechaInscripcion
R11	6	idRoutine → fechaCreacion, estado, refEstructura, idWorkoutProgram, idTrainerCreator, idRoutineOrigen
R12	2	idParameters → idRoutine; idRoutine → idParameters
R13	2	(idRoutine, idExercise) → orden; (idRoutine, orden) → idExercise
R14	5	idValidation → idRoutine, idTrainerEvaluador, fechaValidacion, decision, observaciones
R15	5	idAssignment → idRoutine, idClient, fechaAsignacion, fechaFin; (idRoutine, idClient) → idAssignment
R16	3	idSesion → idAssignment, fechaSesion; (idAssignment, fechaSesion) → idSesion
R17	6	idRequest → idClient, idRoutine, idTrainer, fechaSolicitud, motivo, estado
R18	4	idRequest → fechaRespuesta, descripcionAjuste, idRoutineResultante, idTrainerResolutor
Total		**59**

5.4. Claves candidatas y formas normales

Antes de comprobar las formas normales conviene identificar las claves candidatas, porque de ellas depende el criterio de la 3FN. Varias relaciones tienen más de una clave candidata: `USER`, `SITE`, `WORKOUT_PROGRAM` y `MUSCLE_GROUP` tienen el nombre o el correo como clave alternativa, y `PARAMETERS_ROUTINE`, `CONTAINS`, `ASSIGNMENT` y `SESION` poseen una clave compuesta alternativa por las restricciones de unicidad de la sección 4. El Cuadro 4 las resume.

Para las formas normales se aplicaron estos criterios:

- **1FN:** todos los atributos son atómicos y no hay grupos repetitivos. El atributo `refEstructura` de la entidad `ROUTINE` almacena la estructura interna anidada en formato `JSONB` dentro de PostgreSQL, el cual se trata como un tipo de dato nativo atómico a nivel relacional, preservando la 1FN.
- **2FN:** 1FN y ningún atributo no primo depende de una parte propia de una clave candidata.
- **3FN:** para toda dependencia no trivial $X \to A$ de $F_{\text{mín}}$, X es superclave o A es primo.
- **FNBC:** para toda dependencia no trivial $X \to A$, X es superclave.

Primeras formas normales. Las interrelaciones de cardinalidad máxima N en ambos extremos se resolvieron con relaciones propias, de modo que no quedan atributos multivaluados. La clasificación por tipo e intensidad en `EXERCISE` se gestiona mediante atributos de dominio acotado. Asimismo, las proporciones de tipo de ejercicio en la parametrización se representan como columnas atómicas. La lista de grupos musculares de un programa se externalizó a la relación asociativa `COVERS`. No hay, por tanto, grupos repetitivos y las 18 relaciones están en 1FN.

Segundas y terceras formas normales. Los determinantes de $F_{\text{mín}}$ son, en todos los casos, claves candidatas. No hay dependencias parciales: en las relaciones con clave compuesta (R6, R9, R10, R13, R15, R16 y R18) la clausura de cualquier subconjunto propio de la clave es el propio subconjunto, porque no existe ninguna dependencia cuyo lado izquierdo sea un determinante menor. Tampoco hay dependencias transitivas: como todo determinante es clave candidata, y toda clave candidata determina la relación completa, no existe un atributo no primo que dependa de otro atributo no primo. En consecuencia, las 18 relaciones están en FNBC, forma normal máxima para el modelo de dependencias funcionales, que es más fuerte que la 3FN exigida.

5.5. Propiedad de join sin pérdida de información

Una descomposición es sin pérdida si la reconstrucción por unión natural de sus relaciones produce exactamente las instancias de la relación original. Para comprobarlo se aplican dos herramientas: el Lema de Ullman, para las uniones binarias, y el algoritmo de la tabla, para un caso desarrollado.

Lema de Ullman. El lema establece que la unión de R_1 y R_2 es sin pérdida si y solo si $R_1 \cap R_2 \to R_1 \setminus R_2$ o $R_1 \cap R_2 \to R_2 \setminus R_1$ pertenece a F. En este diseño, como todas las claves foráneas referencian la clave primaria completa de la relación padre, se cumple siempre que $R_1 \cap R_2$ es la clave primaria de la relación hija y que esa clave $\to$ el resto de la relación pertenece a $F_{\text{mín}}$. En efecto, la unión de dos relaciones ligadas por una clave foránea no puede producir tuplas espurias: al ser la clave primaria única y obligatoria, cada fila de la relación hija se empareja con exactamente una fila de la padre. Por tanto, cada unión binaria del diseño es sin pérdida y, por inducción sobre el número de relaciones, lo es toda unión secuencial que siga las claves foráneas.

Un matiz importante: el esquema no se obtiene de una única relación universal de la que se

Cuadro 4: Claves candidatas y minimalidad.

Rel.	**Claves candidatas**	**Justificación de la minimalidad**
R1	idUser; correo	Atributo único en ambos casos; la unicidad del correo es U-1.
R2	idTrainer	Atributo único.
R3	idClient	Atributo único.
R4	idSite; nombre	Atributos únicos; el nombre es único por U-3.
R5	idWorkoutProgram; nombre	Atributos únicos; el nombre es único por U-2.
R6	(idWorkoutProgram, idMuscleGroup)	Clave compuesta completa: la clausura de cada atributo aislado es él mismo, luego es minimal.
R7	idMuscleGroup; nombre	Atributos únicos.
R8	idExercise; (nombre, idMuscleGroup)	Ambas son atributos únicos o clave compuesta minimal; la segunda es el criterio de duplicidad de U-12.
R9	(idTrainer, idWorkoutProgram)	Compuesta y minimal.
R10	(idClient, idWorkoutProgram)	Compuesta y minimal.
R11	idRoutine	Atributo único.
R12	idParameters; idRoutine	Atributos únicos; la segunda se sigue de la unicidad U-8.
R13	(idRoutine, idExercise); (idRoutine, orden)	Ambas compuestas y minimales, por U-6 y U-7: las clausuras de idRoutine e idExercise aisladas no contienen el conjunto completo.
R14	idValidation	Atributo único.
R15	idAssignment; (idRoutine, idClient)	Atributo único y clave compuesta minimal, por U-9.
R16	idSesion; (idAssignment, fechaSesion)	Atributo único y clave compuesta minimal, por U-10.
R17	idRequest	Atributo único.
R18	idRequest	Atributo único; la unicidad U-11 impide que exista más de un resultado por solicitud.

Cuadro 5: Determinantes de $F_{\text{mín}}$ y forma normal alcanzada.

Rel.	**Determinantes de $F_{\text{mín}}$**	**¿Superclave?**	**FN máx.**
R1	idUser; correo	Sí (ambas)	FNBC
R2	idTrainer	Sí	FNBC
R3	idClient	Sí	FNBC
R4	idSite; nombre	Sí (ambas)	FNBC
R5	idWorkoutProgram; nombre	Sí (ambas)	FNBC
R6	(idWorkoutProgram, idMuscleGroup)	Sí	FNBC
R7	idMuscleGroup; nombre	Sí (ambas)	FNBC
R8	idExercise; (nombre, idMuscleGroup)	Sí (ambas)	FNBC
R9	(idTrainer, idWorkoutProgram)	Sí	FNBC
R10	(idClient, idWorkoutProgram)	Sí	FNBC
R11	idRoutine	Sí	FNBC
R12	idParameters; idRoutine	Sí (ambas)	FNBC
R13	(idRoutine, idExercise); (idRoutine, orden)	Sí (ambas)	FNBC
R14	idValidation	Sí	FNBC
R15	idAssignment; (idRoutine, idClient)	Sí (ambas)	FNBC
R16	idSesion; (idAssignment, fechaSesion)	Sí (ambas)	FNBC
R17	idRequest	Sí	FNBC
R18	idRequest	Sí	FNBC

haga una descomposición, sino de 18 relaciones ya construidas a partir del MERX, donde las dependencias de F son intrarrelación. Por eso la PLJ se verifica uniendo cada relación hoja con las relaciones a las que llega por sus claves foráneas. Las hojas del esquema, es decir, las relaciones que ninguna otra referencia, son R2, R6, R12, R13, R14, R18. El Cuadro 6 recoge las cadenas de unión que entre todas cubren las 18 relaciones.

En cada unión de la tabla, la intersección es exactamente la clave primaria de la relación que se incorpora, y esa clave $\rightarrow$ el resto de sus atributos pertenece a $F_{\text{mín}}$. Por tanto, todas las uniones son sin pérdida y cada hoja genera un conjunto ρ_h sin pérdida.

Cuadro 6: Cadenas de unión para la comprobación de la PLJ.

Hoja	**Orden de uniones (por clave primaria completa)**
R2	R2 $\bowtie$ R1
R6	R6 $\bowtie$ R5; R6 $\bowtie$ R7
R12	R12 $\bowtie$ R11 $\bowtie$ R5; R11 $\bowtie$ R2 $\bowtie$ R1
R13	R13 $\bowtie$ R11 $\bowtie$ R8 $\bowtie$ R7; R8 $\bowtie$ R2 $\bowtie$ R1
R14	R14 $\bowtie$ R11 $\bowtie$ R5; R11 $\bowtie$ R2 $\bowtie$ R1; R14 $\bowtie$ R2 $\bowtie$ R1
R18	R18 $\bowtie$ R17 $\bowtie$ (R3, R11, R2) $\bowtie$ R1; R18 $\bowtie$ R11 $\bowtie$ R5

Algoritmo de la tabla (caso desarrollado). Se desarrolla el algoritmo de la tabla de Chase sobre una unión concreta: la que recorre el camino de una validación hasta el nombre del programa de la rutina validada. Se toman las proyecciones relevantes S_1 (de `VALIDATE`), S_2 (de `ROUTINE`), S_3 (de `WORKOUT_PROGRAM`) y S_4 (de `TRAINER`), y se renombran los atributos: v = idValidation, p = idRoutine, t = idTrainerEvaluador / idTrainerCreator, r = decision, l = idWorkoutProgram, n = nombre, e = especialidad.

Las dependencias aplicables son $F = \{v \rightarrow p,\ v \rightarrow t,\ v \rightarrow r,\ p \rightarrow l,\ p \rightarrow t,\ l \rightarrow n,\ t \rightarrow e\}$. El Cuadro 7 muestra la tabla inicial: se coloca el símbolo distinguido a_j en la celda (i, j) si el atributo pertenece a S_i, y un símbolo genérico b_{ij} en caso contrario.

Cuadro 7: Tabla inicial del algoritmo de la PLJ.

	v	p	t	r	l	n	e
S_1	a_1	a_2	a_3	a_4	b_{15}	b_{16}	b_{17}
S_2	b_{21}	a_2	a_3	b_{24}	a_5	b_{26}	b_{27}
S_3	b_{31}	b_{32}	b_{33}	b_{34}	a_5	a_6	b_{37}
S_4	b_{41}	b_{42}	a_3	b_{44}	b_{45}	b_{46}	a_7

El algoritmo itera así:

1. $p \rightarrow l, t$: S_1 y S_2 coinciden en p (a_2), de modo que $b_{15} := a_5$.
2. $l \rightarrow n$: S_1, S_2 y S_3 coinciden en l (a_5), por lo que $b_{16} := a_6$ y $b_{26} := a_6$.
3. $t \rightarrow e$: S_1, S_2 y S_4 coinciden en t (a_3), por lo que $b_{17} := a_7$ y $b_{27} := a_7$.
4. Se alcanza el punto fijo.

Tras la iteración, la fila S_1 queda como $(a_1, a_2, a_3, a_4, a_5, a_6, a_7)$: todos los símbolos distinguidos. Como al menos una fila tiene todos sus símbolos distinguidos, la unión es sin pérdida. Queda verificada la PLJ de forma algorítmica.

5.6. Preservación de las dependencias funcionales

Una descomposición preserva F si $F^+ \equiv \left(\bigcup_{i=1}^{k} \Pi_{R_i}(F^+)\right)^+$, es decir, si toda dependencia del esquema original puede deducirse de las dependencias proyectadas sobre las relaciones resultantes. El algoritmo de comprobación recorre, para cada $X \to Y$ de F, el conjunto $Z := X$ y lo va ampliando con $Z := Z \cup \left((Z \cap R_i)^+ \cap R_i\right)$ para cada relación, hasta punto fijo; la dependencia se preserva si $Y \subseteq Z$.

Por construcción del algoritmo de síntesis, la PPDF está garantizada. Toda dependencia $X \to A$ de $F_{\text{mín}}$ fue declarada dentro de una relación R_i, de modo que $X \cup \{A\} \subseteq R_i$. Como $F_{\text{mín}} \equiv F$, las 59 dependencias de $F_{\text{mín}}$ quedan contenidas en alguna $\Pi_{R_i}(F_{\text{mín}})$ y, por tanto, $F^+ \equiv \left(\bigcup_i \Pi_{R_i}(F^+)\right)^+$.

Se desarrolla el algoritmo sobre una dependencia que cruza relaciones: la que relaciona el identificador de una validación con el nombre del programa ($v \to n$). Tomando $Z := \{v\}$:

1. R_{14} (`VALIDATE`): $Z \cap R_{14} = \{v\}$, obteniendo $v^+ \cap R_{14} = \{v, p, t, r\}$, luego $Z = \{v, p, t, r\}$.
2. R_{11} (`ROUTINE`): $Z \cap R_{11} = \{p\}$, obteniendo $p^+ \cap R_{11} = \{p, l\}$, luego $Z = \{v, p, t, r, l\}$.
3. R_5 (`WORKOUT_PROGRAM`): $Z \cap R_5 = \{l\}$, obteniendo $l^+ \cap R_5 = \{l, n\}$, luego se añade n.

Como $n \subseteq Z$, la dependencia queda preservada a pesar de encadenar tres relaciones mediante claves foráneas. Queda verificada la PPDF.

5.7. Anomalías evitadas

Para poner a prueba el diseño se lo compara con un esquema desnormalizado de referencia, por ejemplo `SESION_PLANO`(idAssignment, idClient, nombreCliente, idRoutine, fechaRutina, fechaSesion, horaInicio, completada), que repite el nombre del cliente y la fecha de la rutina en cada sesión registrada.

Como el esquema está en FNBC, no queda redundancia debida a dependencias funcionales. Existen, no obstante, tres redundancias controladas que no son dependencias intrarrelación y que el diseño protege con disparadores:

- `USER.rol` es derivable de la existencia de la fila en el subtipo correspondiente. Se mantiene porque el sistema necesita filtrar por rol en cada consulta, y se protege con un disparador que exige que *rol = 'entrenador'* si y solo si existe fila en `TRAINER`, e igual para los demás subtipos.
- `ROUTINE.estado` guarda información relacionada con `VALIDATE.decision`. Se protege con un disparador que deriva el estado de la rutina a partir de la última validación registrada.
- El entrenador que resuelve un ajuste coincide necesariamente con el entrenador que el cliente eligió en la solicitud, porque el enunciado pide que sea ese quien ejecute la tarea. La relación `ADJUSTMENT_RESULT` guarda por eso un atributo `idTrainerResolutor` que es derivable de `REQUEST`. Se protege con un disparador que exige la igualdad.

Cuadro 8: Anomalías de inserción, eliminación y actualización.

Anomalía	**Esquema desnormalizado**	**Cómo la evita el diseño**
Inserción	No se puede registrar una sesión sin repetir el nombre del cliente y la fecha de la rutina, ni registrar un cliente sin sesión.	`SESION`, `ASSIGNMENT`, `ROUTINE` y `USER` se insertan de forma independiente; solo las relaciones hijas exigen que exista el padre.
Eliminación	Borrar la última sesión de una rutina eliminaba también el nombre del cliente y la fecha de la rutina, con pérdida de información.	Borrar una fila de `SESION` o de `VALIDATE` solo elimina ese hecho. Las claves foráneas impiden borrar un padre con hijos y la baja de rutinas es lógica, mediante el estado.
Actualización	Cambiar *nombreCliente* obligaba a modificar todas las filas de `SESION_PLANO`; omitir una producía inconsistencia.	El nombre del cliente se almacena una sola vez, en `USER`. Lo mismo ocurre con el objetivo de un programa, el nombre del catálogo o la especialidad de un entrenador.

5.8. Alcance de la validación formal

Cuadro 9: Alcance de la validación formal.

Componente	**Almacén**	**¿Aplican 3FN, PLJ y PPDF?**
Las 18 relaciones del núcleo relacional	PostgreSQL	Sí: es el alcance de esta sección.
Estructura de bloques de la rutina	PostgreSQL (`JSONB`)	No: el campo anidado admite esquemas flexibles en formato documental. Su integridad se rige por las restricciones funcionales y de negocio de la sección 4.9.
Rutina activa en caché	PostgreSQL / Memoria	No: optimización de lectura orientada al rendimiento transaccional.

La normalización y las propiedades PLJ y PPDF están definidas sobre esquemas relacionales puros, por lo que aplican al núcleo transaccional en PostgreSQL. El uso del atributo `refEstructura` como tipo `JSONB` permite almacenar de forma estructurada e indexada los bloques de la rutina sin fragmentar el motor relacional ni requerir sistemas externos adicionales.

5.9. Correspondencia entre requerimientos y relaciones

El Cuadro 10 verifica que cada bloque de requerimientos funcionales e informacionales tiene soporte en el esquema.

Cuadro 10: Correspondencia entre requerimientos y relaciones del esquema.

Requerimientos	**Relaciones que los soportan**
RF-01 a RF-03 (autenticación, roles, cuentas)	`USER`, `TRAINER`, `CLIENT`
RF-04 (entrenadores)	`TRAINER`, `AUTHORISES`
RF-05, RF-06 (clientes y sedes)	`CLIENT`, `SITE`
RF-08 a RF-10 (programas y grupos)	`WORKOUT_PROGRAM`, `COVERS`, `MUSCLE_GROUP`
RF-11 a RF-15 (banco de ejercicios)	`EXERCISE`, `MUSCLE_GROUP`
RF-16 a RF-19 (autorización e inscripción)	`AUTHORISES`, `INSCRIBED`
RF-20 a RF-24 (generación y composición)	`ROUTINE`, `PARAMETERS_ROUTINE`, `CONTAINS`
RF-25 a RF-27 (validación)	`VALIDATE`, `ROUTINE`
RF-28 a RF-31 (asignación y sesiones)	`ASSIGNMENT`, `SESION`
RF-32 a RF-34 (solicitudes de ajuste)	`REQUEST`, `ADJUSTMENT_RESULT`
RF-35 a RF-42 (consultas y reportes)	`ROUTINE`, `SESION`, `VALIDATE`, `EXERCISE`, `CONTAINS`
RF-43 a RF-45 (presentación y exportación)	Todas; dependen de la aplicación, no del esquema
RI-01 a RI-04	`USER`, `TRAINER`, `CLIENT`, `SITE`
RI-05 a RI-09	`WORKOUT_PROGRAM`, `COVERS`, `MUSCLE_GROUP`
RI-10 a RI-13	`EXERCISE`
RI-14, RI-15	`AUTHORISES`, `INSCRIBED`
RI-16 a RI-19	`ROUTINE`, `PARAMETERS_ROUTINE`, `CONTAINS`
RI-20, RI-21	`VALIDATE`
RI-22 a RI-24	`ASSIGNMENT`, `SESION`
RI-25 a RI-27	`REQUEST`, `ADJUSTMENT_RESULT`
RI-28 a RI-35	`ROUTINE`, `SESION`, `VALIDATE`, `EXERCISE`

5.10. Criterios de evaluación aplicados

Cuadro 11: Criterios de evaluación del diseño.

Criterio	**Evaluación**	**Justificación**
Normalización	Correcta (FNBC)	Las 18 relaciones están en FNBC, con 3FN, PLJ y PPDF verificadas.
Integridad referencial	Completa	Las claves foráneas están declaradas y las reglas de negocio cubren los casos críticos.
Trazabilidad	Alta	El historial de ejercicios, rutinas, validaciones, asignaciones, sesiones y ajustes es reconstruible.
Flexibilidad	Adecuada	`PARAMETERS_ROUTINE` separada, `CONTAINS` con *orden* y `JSONB` permiten evolucionar sin migrar esquemas rígidos.
Escalabilidad	Diseñada	El almacenamiento unificado en PostgreSQL con índices eficientes evita degradar el núcleo transaccional.
Correspondencia con el dominio	Alta	Cada requerimiento funcional e informacional tiene contraparte en el esquema.

5.11. Riesgos y mitigaciones

5.12. Conclusión de la valoración

El diseño lógico obtenido a partir del MERX es correcto:

- Las 18 relaciones están en 1FN, 2FN, 3FN y, en términos de fuerza normal, en FNBC.
- La descomposición cumple la propiedad de join sin pérdida, verificada con el Lema de Ullman en cada unión por clave primaria y con el algoritmo de la tabla en el caso desarrollado.
- La descomposición preserva las dependencias de $F_{\text{mín}}$.
- El diseño evita las anomalías de inserción, eliminación y actualización; las redundancias controladas restantes están cubiertas por disparadores.
- Cada requerimiento funcional e informacional tiene soporte en el esquema.

Por tanto, el diseño es válido y cumple las condiciones necesarias para implementarse en un SGBD relacional como PostgreSQL.

Cuadro 12: Riesgos del diseño y mitigaciones previstas.

Riesgo	**Impacto**	**Mitigación**
Duplicidad de ejercicios no detectada	Inconsistencia del banco	Restricción U-12 con índice único sobre (nombre, idMuscleGroup) y validación en la aplicación.
Asignación doble de rutinas	Conflictos operativos	Restricción U-9 (`UNIQUE` sobre (idRoutine, idClient)) y control transaccional.
Desincronización en consultas complejas	Mostrar una rutina u operación obsoleta	Control transaccional estricto en PostgreSQL sobre el núcleo relacional.
Petrificación de los catálogos	Imposibilidad de añadir un tipo o sede	Las clasificaciones principales son tablas, no columnas enumeradas: añadir un valor es insertar una fila.
Derivación del estado de la rutina	Incoherencia entre *estado* y *decision*	Disparador (*trigger*) que recalcula el estado en cada validación.
Pérdida de trazabilidad de la estructura de bloques	Auditoría incompleta	Referencia `refEstructura` obligatoria y sin nulos, validada mediante esquemas `JSONB` estructurados.

6. Conclusiones

El presente informe documenta el análisis, el diseño y la valoración teórica de la base de datos del sistema de generación automática de planes de entrenamiento. El trabajo permitió convertir la narrativa del enunciado en un conjunto de 45 requerimientos funcionales, 12 no funcionales y 35 informacionales, cada uno verificable y trazable, y a partir de ellos identificar las entidades, atributos e interrelaciones del dominio.

El MERX se organizó en una única figura que, sin repetir entidades ni atributos, reúne los 18 conjuntos de entidades, las interrelaciones del dominio y la generalización de *User*. El paso de ese modelo al esquema relacional produjo 18 relaciones, 28 claves foráneas y una decisión de modelado deliberada: el papel de administrador no se materializa como entidad, sino como valor del atributo *rol*, porque sus responsabilidades se traducen en restricciones de unicidad sobre

entidades que ya existen. Esa misma lógica se aplicó en la sección 5 al conservar solo una de las dos vías posibles hacia el cliente de una sesión: incluirla habría producido una dependencia transitiva y habría impedido alcanzar la tercera forma normal.

La especificación de integridad se organizó en cinco familias (entidad, dominio, referencial, unicidad y nulabilidad), más las restricciones de cardinalidad, las semánticas y de negocio, las temporales y los mecanismos de implantación. Cada restricción indica el mecanismo concreto que la implementa, lo que permite derivar el esquema físico sin pasos intermedios.

La valoración formal confirmó que el esquema cumple la tercera forma normal, la preservación de las dependencias funcionales y la propiedad de join sin pérdida de información, apoyándose en el Lema de Ullman para las uniones por clave primaria y en el algoritmo de la tabla para un caso desarrollado. Se comprobó también que el diseño evita las anomalías clásicas y que cada requerimiento del enunciado tiene soporte en el modelo.

Un hallazgo relevante del proceso fue el tratamiento de la estructura variable de bloques. El enunciado pide que no se modele junto al resto de las tablas relacionales convencionales, y el diseño lo respeta unificando la persistencia bajo un único SGBD relacional en **PostgreSQL**: la entidad `ROUTINE` incorpora un atributo `refEstructura` de tipo `JSONB` que almacena de forma estructurada e indexada la jerarquía completa de bloques de cada rutina. Esta decisión evita la fragmentación con motores documentales externos, simplifica la arquitectura y facilita el cumplimiento del requerimiento de disponibilidad inmediata del dato activo mediante índices nativos.

El diseño propuesto constituye así una base sólida y teóricamente robusta para las etapas posteriores de construcción del esquema físico, implementación, carga del conjunto de datos de prueba y pruebas del sistema.